

•



- [Core Concepts](#)
- [Workflows](#)
- Plans in workflows

On this page

Plans in Workflows

Was this page helpful? 

At the [runtime stage of your project](#), the main purpose of your [workflow](#) is to analyze each event in order to produce decisions on the event, for example detecting whether it represents a fraudulent transaction or not. Although the raw event does not contain enough information, you can apply [deployable plans](#) to derive features and transform the input into a very rich schema containing all information about a card, merchant, location, etc.

Still, it is not possible for a machine learning [model](#) to depict fraudulent behavior based only on transformations derived from the raw information contained in the transaction. Hence the need for more than single-moment information about transactions. You must build a card profile, based on daily, weekly, monthly transaction behaviors and collect all the information about location, merchants, and acquirers. Thus, understanding how to calculate average amounts, a sum of amounts, and maybe more complex patterns - aggregated for different periods of time - is also essential.

So, such a [plan](#) specifies how live data must be transformed, handled and prepared to be consumed by the machine learning model, for runtime scoring. When you execute a plan, Pulse translates it into [Pulse Query Language](#) that generates new augmented events ([instances](#)).

With such a Plan you can include the following types of information into runtime events and turn them into instances:

- **Metrics:** Time-based aggregations over data.
- **Data Enrichment Fields:** Fields consisting of data enrichment functions.
- **Fields:** [Fields](#) from the [schema](#) directly added into the model.
- **Features:** Combinations of metrics and/or schema fields that add value to the model.

Metrics

Metrics allow defining aggregations over multiple data rows, covering events over a given [time window](#) (e.g.: the *sum of the amount of transactions in the last month*).

Consider the scenario where the sum of some card's transacted amount per month is approximately \$10,000. It would be unusual if a single transaction exceeded this value; that could be considered as potential fraud. Other examples of metrics that could be used in this case are *minimum amount in last 24 hours*, or *number of transactions per week*. Metrics like these can help establishing a more complete picture of the card's behavior.



info

For details about metrics in the context of the Railgun engine, refer to the [Metrics in Railgun](#) page.

Components of a metric

Defining a metric implies the definition of the following components:

Time window

A **Window expression** can be defined to specify the size of the time window that a metric covers. The expression should be in the form `<number> <period>` ; for example, "4 weeks".

The following table lists the supported period units for window expressions, and the corresponding durations:

Term	Duration (ms)
ms / millis / millisecond / milliseconds ⁽¹⁾	1

Term	Duration (ms)
s / sec / secs / second / seconds	1000
min / mins / minute / minutes	60,000 (60 seconds)
hour / hours	3,600,000 (60 minutes)
day / days	84,400,000 (24 hours)
week / weeks	604,800,000 (7 days)

(1) Milliseconds are the base unit of window duration. All other window duration values are relative to one millisecond.

In addition to the duration, windows can have a [Filter](#) expression, which can be used to limit which events within the window are included in the metric. Filters can only use information about the current event.

Time windows can have different behaviors depending on their type. Refer to the [Glossary](#) for an overview of the various types of time windows available.

Aggregation function¹

The core of a metric is the **Aggregation** function, which determines how to aggregate the data within the window, i.e how to combine the multiple data points into a single value.

Supported aggregations are **avg**, **count**, **max**, **min**, **sum**, as well as advanced expressions implemented in [UDFs](#).

Check the [Aggregations](#) page for aggregation best practices.

Grouping fields¹

One or more grouping fields (or grouping entities) can be provided to cluster the data in buckets that share the same values in those fields. This means that the metric will calculate the aggregations for each cluster and return the results for the cluster that the current event belongs to.

Only strings or categorical variables can be used as grouping entities.

Timestamp¹

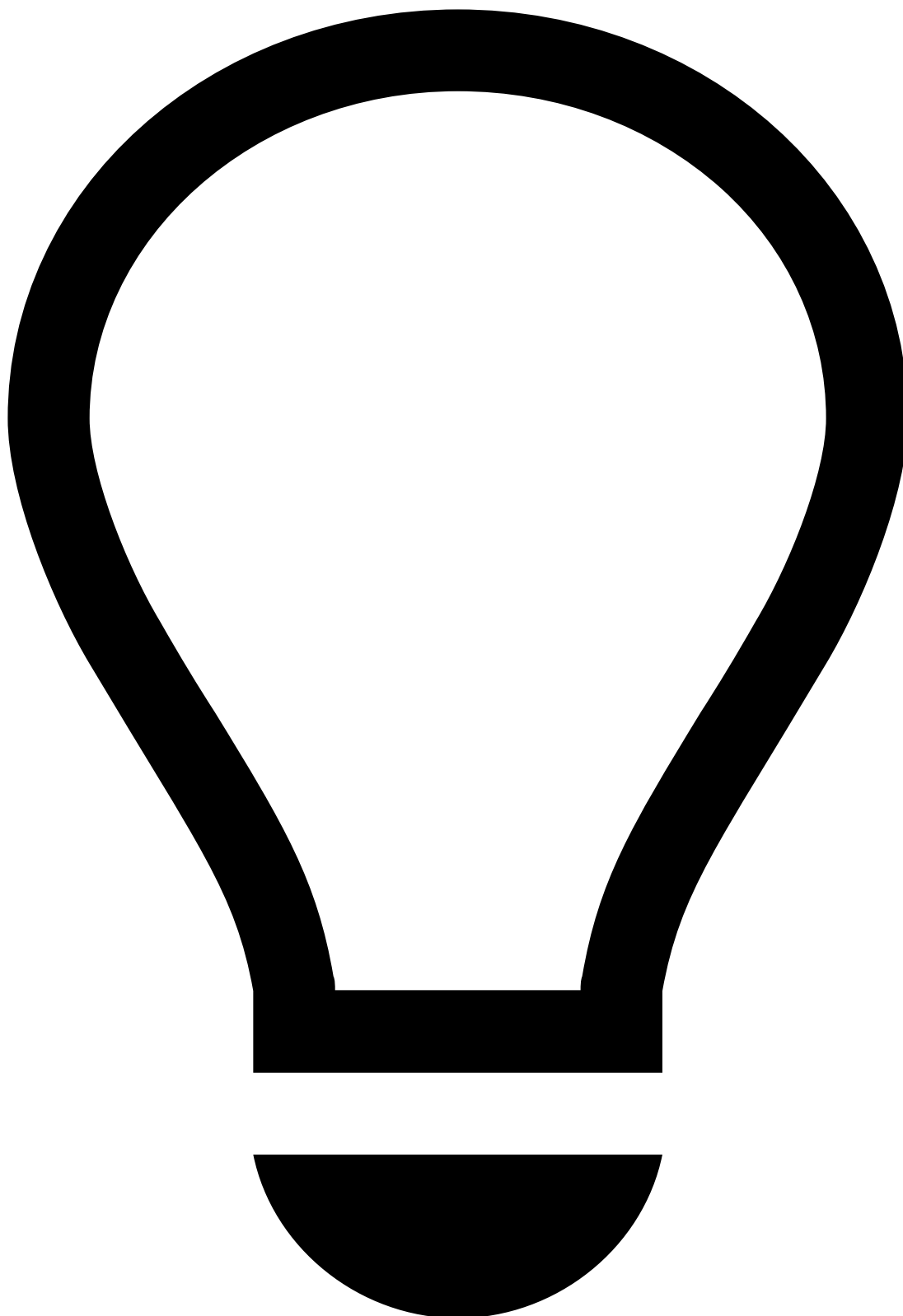
In a plan service of a [workflow](#), Pulse must use the event's timestamp to perform the required time-based aggregations for the calculation of metrics. Pulse ensures that the timestamp is used consistently whether you are performing data science tasks or the plan is deployed into a workflow.

When you design your [plan](#), if you need to calculate metrics, the input [data source](#) must contain a timestamp field. However, to ensure that the runtime behavior does not change after deployment, Pulse uses the [workflow clock](#) (that always keeps the value of the latest timestamp submitted to the workflow) to assign a timestamp value to the event.

This means that, in a workflow, the data schema of your runtime events does not require an explicit timestamp field.

When a plan service needs to calculate metrics, Pulse enforces that events are consistently ordered in time. So, the event's timestamp is internally interpreted as the workflowclock value (the latest timestamp available), in case that event's timestamp is lower. For more information, see [Time, schedules and timezones](#).

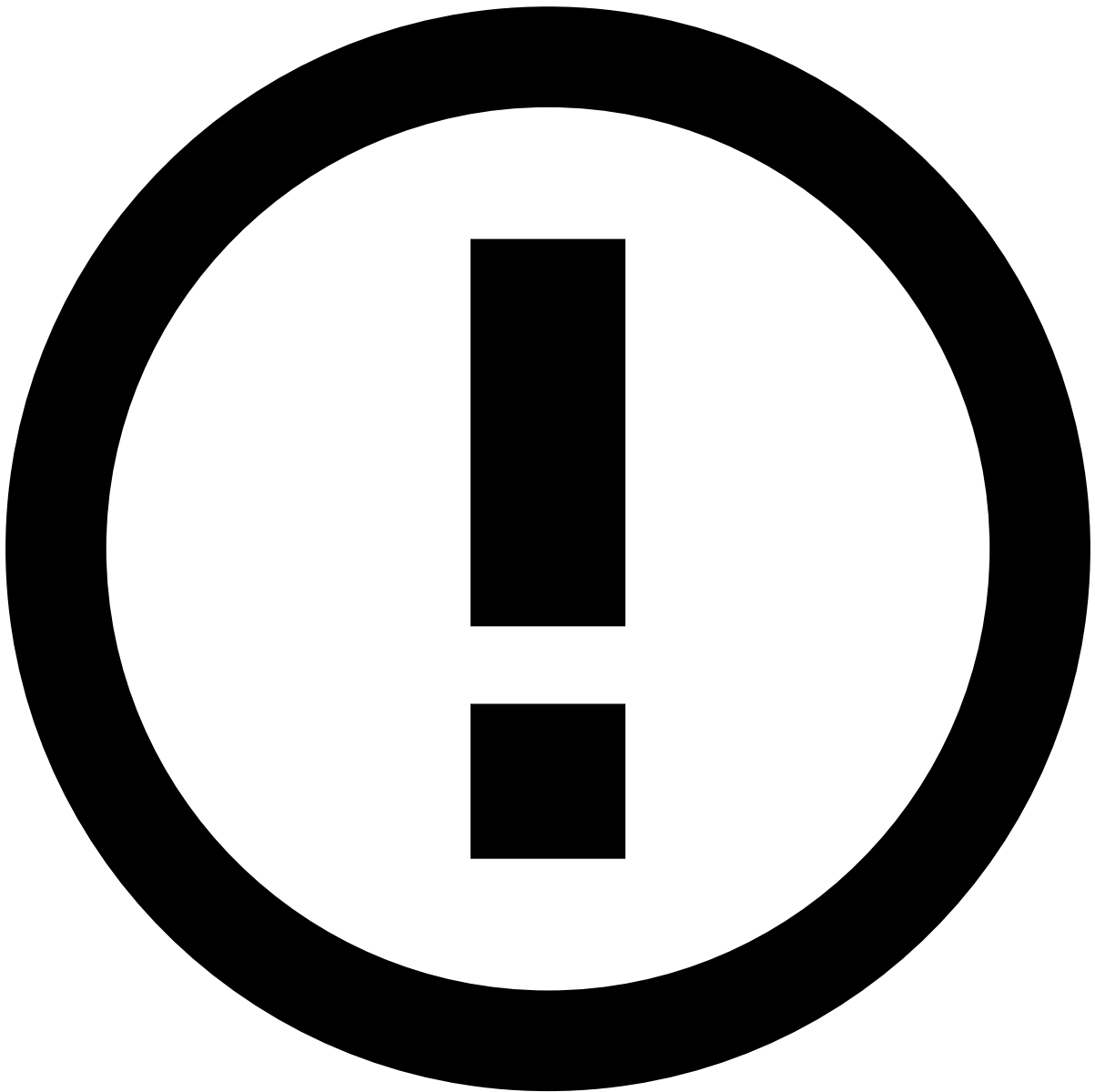
If the workflow pushes events with inconsistent timestamps to a plan service, for example, because of parallel processing of incoming events, the generated profiles may be ignoring events that entered the workflow first and will not reflect the reality.



Section not applicable to Pulse with Railgun

When Pulse is configured with the [Railgun engine](#), it is able to calculate all metrics in real time. Therefore, the concept of scheduled metrics is not relevant, and the limitations of real-time metrics below do not apply.

Metrics in Pulse can be of one of two different types: real-time metrics and scheduled metrics. While both allow storing aggregated data based on your transactions, there is a fundamental difference between them.



Restrictions

If you define more than one metric for your model you must choose at least one common *Group By* field between these metrics.

Real-time metrics in memory

Real-time metrics are updated for each transaction that traverses your [workflow](#). Each transaction will enter Pulse's high-performance event processing engine and within less than milliseconds, your metrics are updated and stored in memory.

These metrics are useful to track short-term behaviors such as a running sum of how much was spent in the past 24 hours. These metrics should be used judiciously for cases where immediate metric updates are desirable.



Precaution with Real-Time Metrics

There are performance and sizing issues to consider while creating real-time metrics. These metrics are being aggregated in real time, collecting events to fill up an in-memory window. All the information is being saved in memory.

Knowing your data rate and schema, you can calculate the amount of space needed to hold the necessary events for the defined window. Take a look at the [table of data types and sizes](#) and estimate the data size to make sure your system is capable of performing what you need.

You can also define real-time metrics which instead of being updated on every event that enters the workflow, are updated every X amount of time defined by the user.



Precaution with Real-Time Metrics

The time interval between every output is only started after an event enters the workflow, not when the workflow is published.

Note that this metric type is exclusive, meaning that it will not include an event that occurred at the same time the interval window expires.

For example, suppose you have a 5 day window which is updated every hour that will count the number of transactions done by a credit card number.

Let's assume for that an event occurs on Monday at midnight which marks the start of the time interval before outputting the metric.

The following events then occur:

Timestamp	Value	Metric value
00:05	(cardNumber: 1, transactionValue: Â£10)	Count(cardNumber: 1) = 0
00:27	(cardNumber: 2, transactionValue: Â£36)	Count(cardNumber: 1) = 0
00:42	(cardNumber: 1, transactionValue: Â£42)	Count(cardNumber: 1) = 0
01:00	(cardNumber: 1, transactionValue: Â£140)	Count(cardNumber: 1) = 0
01:01	(cardNumber: 2, transactionValue: Â£5)	Count(cardNumber: 1) = 2
02:00	(cardNumber: 3, transactionValue: Â£23)	Count(cardNumber: 1) = 3

Real-time metrics on disk

In a production environment, Pulse [can be configured](#) to store the state of real-time metrics on disk. This allows Pulse to calculate metrics over larger time windows. However, be aware of the limitations for some aggregations in the [Creating metrics](#) page.



Limitations

Note that this feature is in continuous development, and as of now it does not provide full feature parity to the previous, RAM-based version.

Scheduled metrics

Scheduled metrics are defined in the same way as their real-time counterparts but they are updated as a batch process from time to time in a user-defined schedule, hence the name.

In order to achieve this, each transaction that traverses your [workflow](#) is persisted to a database. Eventually, when the scheduling criteria are met (i.e. at the end of the month or end of the week), a batch process reads transactions from the database, computes the metrics using Pulse's high-performance event processing engine and stores them to a data store. Scheduled metrics eventually expire from the data store after the time window passes.

Each workflow service has its own implementation of scheduled jobs, depending on its type and its configuration, and may or may not have scheduled jobs to run.

In the specific case of [deployable plans](#), you should define scheduled metrics using the [Metrics operator](#).

When such plans are deployed to a [workflow](#) and configured to run in a plan service, Pulse generates one scheduled metrics job for each scheduled metric in the Plan.



Metrics performance

In order to maintain high performance when calculating metrics, it is recommended that each of your deployable plans contains only one Metrics operator.

To ensure the consistency of the output metrics, whenever processing reaches a Metrics operator Pulse must always keep the context of all calculations performed in previous Metrics operators.

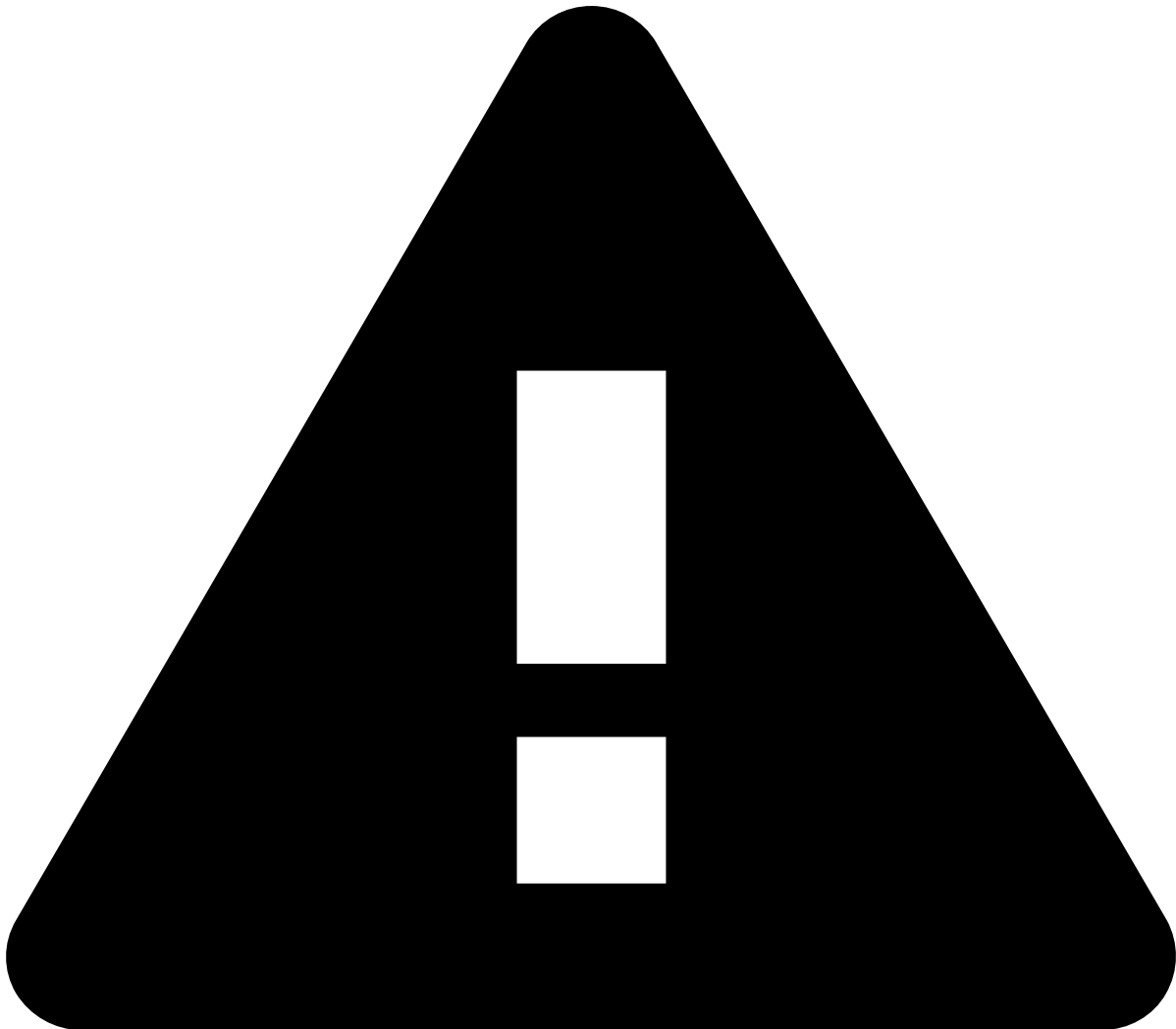
To better understand how scheduled metrics relate to Pulse's job management model, check [how scheduled metrics jobs work](#).

Omnichannel metrics⚠️

To compute metrics based on information from multiple [channels](#), you can use [omnichannel plans](#):

- The **main channel** of the plan corresponds to the events that are processed and scored by the workflow.
- **Secondary channels** of the plan correspond to events of other workflows. You will need to specify which workflow to use for each secondary channel when adding an omnichannel plan to the workflow.

For real-time metrics that use information from multiple channels, events from other workflows will be automatically fed to the plan in the current workflow in real time.



caution

Real-time metrics are not yet supported in omnichannel plans.

However, with [Railgun](#) it is possible to use a subset of omnichannel metrics, called [cross-channel metrics](#).

For scheduled metrics, the batch job will process data from multiple workflows automatically â€” check [how scheduled metrics jobs work](#) for more information.

When an omnichannel plan in workflow A is processing an event from the secondary workflow B, an important aspect is: which fields does that event include? By default, the plan will receive the event as it entered workflow B, i.e. without any fields added by workflow B. This might not be ideal, if workflow B is modifying some of the fields to standardize the values, or is adding useful computed fields that workflow A needs to access. This problem can be solved with a configuration

property, `workflowReplicaSchemaMarker` . In the example above, you can set this property on the workflow B to mark a point in the workflow (an identifier of a workflow service) at which all the necessary fields have been added or modified. The event will then be shared with workflow A in the same state as when it leaves the marked service.

Data enrichment📖

Pulse's algorithms are integrated with federated data sources such as databases of TOR nodes, compromised IP addresses, malicious emails, etc, to understand if a transaction is related to any of these problematic resources, and thus becoming a suspicious transaction.

Adding these fields to your events raises the probability of obtaining better results, for example for fraud detection.



Feedzai IQ

[Feedzai IQ](#) provides an additional set of federated risk insights that can be used in your workflows, leveraging aggregated financial risk intelligence from the Feedzai global network.

Fields

Fields are the transaction attributes that are passed directly into the instance (i.e. the event that is actually sent to machine learning algorithms). These fields form a subset of the features/instances dataset schema.

For more details on the possible configurations for fields, check the Pulse [data types](#) reference.

Categorical fields

Categorical fields are fields whose values are represented by a set of different values that belong to the same category. For example, a fraud field will have two possible values: **true** or **false**. A country field will have a list of possible values describing country names.

This means that these categorical fields are marked as dimensions and their values are represented by dimension mappings. Make sure that categorical fields have all the necessary dimension mappings.



Caution with high cardinality fields

Be careful when using categorical fields with a high number of dimension mappings. If the cardinality is too high, it might be that some values are never observed during training.

A model that uses these fields will generally result in poor classifications when an instance with such a value is scored. To avoid this problem, create a custom feature and group the field values into buckets that are large enough (e.g. if your system supports hundreds of operation types you should consider grouping them into common categories such as *withdrawal*, *transfer*, *payment*, *query*, etc).

Missing values❗

In a long-term deployment, you must have resilience to errors. When receiving an event with an unknown value for a categorical field (maybe due to a new currency or operation type) or no value at all (perhaps caused by faulty equipment) that can lead to misleading scoring results.

During the Data Science stage, and whenever you execute a task that generates a [data source](#), for example, as the output of running a [plan](#), you can define how to represent missing values. Pulse sets the value as, for instance, NULL, so that in practice it is no longer missing.

Unlike Data Science, where you know the entirety of your datasets, in runtime, you cannot predict if and when data is incomplete or invalid. Additionally, if you have designed features and metrics that depend on a field value that Pulse cannot process, you would allow errors to propagate through your [workflow](#).

Depending on your scenario you must define a strategy to ensure that Pulse can score these instances, as part of a [data cleaning](#) pre-processing, and/or with custom logic defined at custom workflow services.



This usually involves setting as string the fields that you expect to have some volatility in terms of possible values, for example, *country*.

If you configure a field with the [string type](#), instead of a categorical, Pulse recognizes all values as valid. You will lose the ability to perform Data Science operations directly over the field. However, you can define a plan to remap string values back to categories.

Possible strategies are:

- **Defining a fallback value:** For instance, if the system operates mostly on US transactions you could assume that when an event arrives with an unknown country field, it should be treated as coming from the US.
- **Assigning an "undefined" or "unknown" value to the field:** This alternative explicitly creates an extra category to catch all instances that Pulse would not be able to score. However, you must be aware that you will be scoring instances that may not have all the necessary information for the [model](#) to produce an accurate result. You must ensure that features that depend on the field are not incorrectly generated.
- **Discarding the event:** If you don't handle the unknown values, then Pulse will discard these instances and not score them.

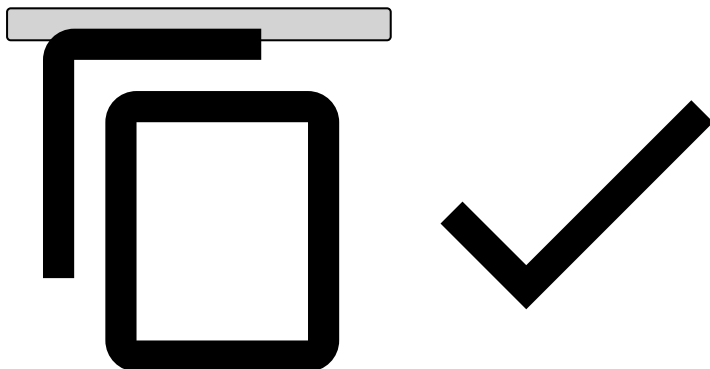
Features

Features allow combining different [metrics](#) and/or perform data transformations over the raw fields of an transaction.

Considering a [model](#) containing a field that indicates whether an operation was performed in a certain country (e.g. a domestic transaction in banks), you can create a feature derived from existing fields. In this case, it would be based on the expression that evaluates to **true** when the country code belongs to the **domestic** country.

For example:

```
(if (cardcountry == "USA") then 1 else 0)
```

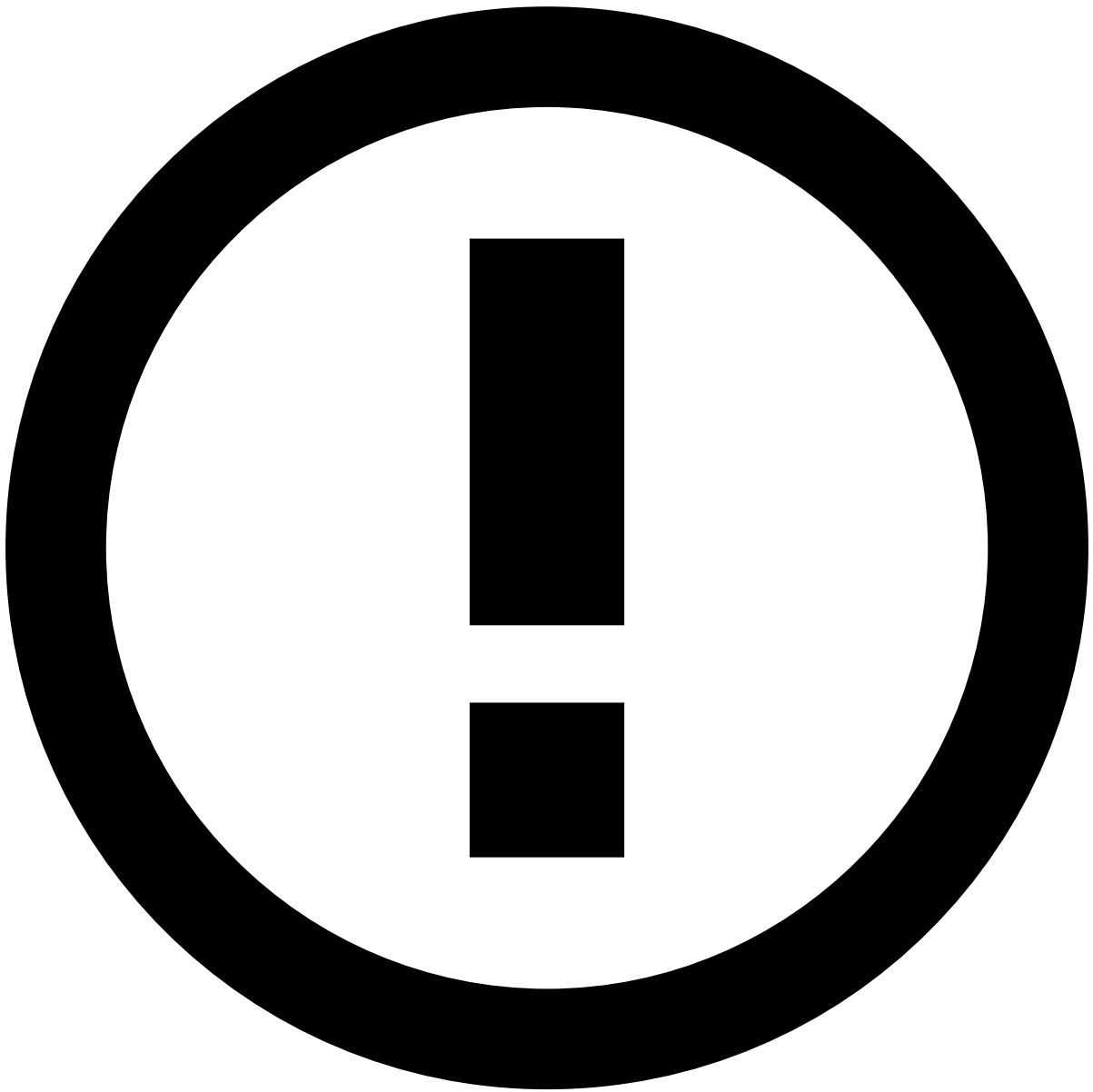


Consider also that you want to derive a feature from metrics: the average transaction amount by country, for the last 24 hours. You can create the following:

- Two metrics - the *total amount by country* and *total transactions by country* for last 24 hours.
- A feature based on the metrics where you divide the first metric by the second one.

The following pages provide practical guides explaining how to add features to your dataset:

- [Creating Derived Features](#)
- [Creating Metrics](#)



Missing values

When specifying the expression you should be aware that the fields and metrics can contain missing values and you need to handle them accordingly. If the user-provided expression does not handle these cases a missing value will be used in its place.

Learn More 

Check the [Workflow concept page](#) for all the details on workflows.